

Class 08

Stored Procedures, Functions & Indexes

◆ 1. Stored Procedures

A **Stored Procedure** is a **predefined set of postgresQL statements** stored in the database that can be executed multiple times.

Think of it as:

| “ A “saved function” for performing repetitive tasks like inserting, updating, or processing data.

Create a Procedure

```
DELIMITER //  
CREATE PROCEDURE getAllStudents()  
BEGIN  
    SELECT * FROM students;  
END //  
DELIMITER ;
```

Explanation:

- DELIMITER — temporarily changes the end symbol to // (so postgresSQL doesn't stop at ; inside the procedure).
- BEGIN...END — defines the block of postgresSQL statements.

Execute Procedure

```
CALL getAllStudents();
```

Procedure with Parameters

1. IN Parameter (input only)

```
CREATE PROCEDURE getStudentByDept(IN dept_name VARCHAR(50))  
BEGIN  
    SELECT * FROM students WHERE department = dept_name;  
END;
```

2. OUT Parameter (returns a value)

```
CREATE PROCEDURE getTotalStudents(OUT total INT)
BEGIN
  SELECT COUNT(*) INTO total FROM students;
END;
```

3. INOUT Parameter (input + output both)

```
CREATE PROCEDURE updateAndReturn(INOUT counter INT)
BEGIN
  SET counter = counter + 1;
END;
```

Control Flow in Procedures

Procedures can include conditions and loops.

```
CREATE PROCEDURE checkGrade(IN marks INT)
BEGIN
    IF marks >= 90 THEN
        SELECT 'A Grade';
    ELSEIF marks >= 75 THEN
        SELECT 'B Grade';
    ELSE
        SELECT 'C Grade';
    END IF;
END;
```

◆ 2. User-Defined Functions (UDFs)

A **User-Defined Function** is similar to a procedure but **returns a single value** and is used directly in PostgreSQL queries.

Create a Scalar Function

```
CREATE FUNCTION getDiscount(price DECIMAL(10,2))  
RETURNS DECIMAL(10,2)  
DETERMINISTIC  
BEGIN  
    RETURN price * 0.10;  
END;
```

Usage:

```
SELECT name, getDiscount(price) AS discount FROM products;
```

Note:

- **Scalar Function** → returns single value.
- **Table-Valued Function** → returns a table (depends on SQL dialect).

◆ 3. Indexes

Indexes speed up data retrieval — just like an **index page in a book** helps you find information quickly.

Without an index, postgresQL scans every row (called a *full table scan*).

With an index, postgresQL jumps directly to relevant rows.

Creating an Index

```
CREATE INDEX idx_student_name  
ON students(name);
```

🗑 Dropping an Index

```
DROP INDEX idx_student_name ON students;
```

Clustered vs. Non-Clustered Index

Type	Description
Clustered Index	Physically arranges table data (usually on Primary Key).
Non-Clustered Index	Creates a separate structure pointing to data locations.

◆ 4. Triggers

A **Trigger** is an automated action that runs **in response to events** like INSERT, UPDATE, or DELETE.

Think of it as:

| “A “listener” that reacts whenever a change happens in the table.

Example: Create a Trigger

```
CREATE TRIGGER before_student_insert
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
  SET NEW.created_at = NOW();
END;
```

Automatically sets `created_at` timestamp when a new student is inserted.

Trigger Timing Options

Timing	Description
BEFORE	Trigger runs before the main query executes.
AFTER	Trigger runs after the query completes.

Common Use Cases

- Automatic audit logs (`INSERT` in log table).
- Enforcing business rules (like preventing negative salaries).
- Maintaining consistency (e.g., updating totals automatically).